Category C Problems

5 problems · steps 1–4 of the walkthrough loop · repeat the core pattern across problems so it sticks · OOP, Classes and Objects — Week 3

L1. Simple Student Record Class

Scenario

The coding club wants the simplest possible way to store one student's name and marks together, instead of two loose, unrelated variables.

Task

- Define a class Student with two fields: String name and int marks.
- In main, create one Student object and set both fields directly.
- Print a single line in the form “Name: ... | Marks: ...”.
- No constructor or extra methods needed for this one — just a class, an object, and field access.

Suggested method signature(s)

```
class Student { String name; int marks; }
```

Sample Input / Output

Input	Output
name = "Divya" *marks = 88*	*Name: Divya | Marks: 88*

Concepts covered: *Class definition, object creation with new, direct field access.*

L2. Two Objects, Two Separate States

Scenario

A hostel warden's whiteboard app tracks the occupied bed count for a room. The club wants to confirm on paper that two room objects never interfere with each other's numbers.

Task

- Define a class HostelRoom with fields String roomNo and int occupied.
- Create two HostelRoom objects, roomA and roomB, with different roomNo values.
- Increase occupied for roomA only, three separate times.
- Print both rooms' occupied values to show roomB was never touched.

Suggested method signature(s)

```
class HostelRoom { String roomNo; int occupied; }
```

Sample Input / Output

Input	Output
roomA = "C-101", roomB = "C-102" occupied on roomA incremented 3 times	*C-101 occupied: 3* *C-102 occupied: 0*

Concepts covered: *Object creation, independent instance state, field access.*

L3. Reading Java's Default Field Values

Scenario

A new club member insists an unset String field starts out as an empty string "". Settle the debate by printing an object's fields right after it's created.

Task

- Define a class LibraryCard with fields String holderName, int booksIssued, and boolean active.
- Create one LibraryCard object using new, without setting a single field.
- Print all three fields immediately, before assigning anything to them.

Suggested method signature(s)

```
class LibraryCard { String holderName; int booksIssued; boolean active; }
```

Sample Input / Output

Input	Output
(no fields set after creation)	*Holder: null* *Books Issued: 0* *Active: false*

Concepts covered: *Java's default field values, object creation, print debugging.*

L4. Basic Constructor for a Course

Scenario

Typing two separate assignment lines for every new Course object is getting tedious fast. Fix it with the simplest possible constructor.

Task

- Define a class Course with fields String code and String title.
- Write a constructor that takes both values as parameters and assigns them using this.code = code; and this.title = title;.
- Create one Course object through the constructor and print its two fields on one line.

Suggested method signature(s)

```
public Course(String code, String title)
```

Sample Input / Output

Input	Output
code = "21CSC201J" *title = "Data Structures"*	*21CSC201J - Data Structures*

Concepts covered: *Constructor basics, this.field = parameter, object creation.*

L5. Counting Objects With a Static Field

Scenario

The admissions desk wants a running count of how many Applicant objects have been created so far this session — one shared number, not something each object tracks separately.

Task

- Define a class Applicant with a static int totalApplicants field, starting at 0.
- In the constructor, increment totalApplicants by 1 every single time.
- Create three Applicant objects.
- Print Applicant.totalApplicants exactly once, accessed through the class name.

Suggested method signature(s)

```
static int totalApplicants;
```

Sample Input / Output

Input	Output
3 Applicant objects created	*Total applicants: 3*

Concepts covered: *Static fields, constructors, accessing static members through the class name.*